al gets byte computation ⇒ do back same one... "lookup"

[0x2b]

(see other sheet of notes)

let $\alpha(x) =$ byte @ ds:[ebx + x]

.loop1 $\left\{ \begin{array}{l} al_{n+1} = \alpha(al_n) \\ dl_{n+1} = \alpha(\alpha(dl_n)) \end{array} \right\}$

US

.loop2 $\left\{ \begin{array}{l} al_{n+1} = \alpha(al_n) \\ dl_{n+1} = \alpha(dl_n) \end{array} \right\}$

al, dl stay together

```
        xor     eax, eax        eax, edx := 0
        xor     edx, edx
.loop1:
        xlatb
        xchg    rax, rdx
        xlatb
        xlatb
        xchg    rax, rdx
        cmp     al, dl
        jnz     .loop1

        xor     eax, eax
.loop2:
        xlatb
        xchg    rax, rdx
        xlatb
        xchg    rax, rdx
        cmp     al, dl
        jnz     .loop2
```

3-lookup chain

2-lookup chain

look to find SAME (al = dl) byte value

rdx | edx | dl

0x2b | rax | eax | al

64 | 32 | 8

note: ~~al ∈ rax~~

~~al ∈ rdx~~

```
.loop1:
xlatb
xchg  rax, rdx
xlatb
xlatb
xchg  rax, rdx
cmp   al, dl
jnz   .loop1
```

xor eax, eax eax := 0

xor edx, edx edx := 0

al := ds:[ebx + (ds:[ebx])] for first table lookup...

"table lookup translation" etc

al := ds:[ebx + al]?

al ~ 0-2

xlatb sets AL := ds:[(e)bx + unsigned al]

locating byte ~~identity~~ in table of memory

— indexed by al — to al

al := ds:[ebx + al] ⟹ al₁ := ds:[ebx + 0] = ds:[ebx]

dl ~ 0-4, al ~ 0-1

⇒ rax := 0s, rax ⊇ dl := ds:[ebx]

al := ds:[ebx + a]

al := ds:[ebx + (ds:[ebx + (ds:[ebx])])]

al := dl := 0

al ∈ @ string

al₂ := ds:[ebx + al₁] ⇒ al := ds:[ebx + 0] = ds:[ebx]

al₃ := ds:[ebx + al₂] ⇒ al := ds:[ebx + (ds:[ebx])]

recurse calls

al ~ dl

⇒ dl ~ 0-4, al ~ 0-1 ⇒ al := ds:[ebx], dl := ds:[ebx + (ds:[ebx])]

root

al ~ 0 ds:[ebx] ≟ ds:[ebx + (ds:[ebx])] al ~ 0-1 recurse calls

al == dl? —Y→ EXIT loop1

exponential difference ⟹ exponential steps: non-linear

~~mov eax, eax eax := 0~~

```
.loop2:
xlatb
xchg  rax, rdx
xlatb
xchg  rax, rdx
cmp   al, dl
jnz   .loop2
```

pass 1

"grab index" al := ds:[ebx]

"save" al := 0?, dl := ds:[eax]

"grab (idx)" al := ds:[ebx] + d]

al := ds:[ebx + d]

al := 0?

pass 2 ds:[eax]

al := ds:[eax

pass 3

al := ds:[ebx + (ds:[ebx] + ds:[ebx + d])]

dl := ds:[ebx + (ds:[ebx + d])]

al := ds:[ebx + (ds:[ebx + d])] = al

is loop1: al := ds:[ebx], dl := ds:[ebx + (ds:[ebx])]

0-1 ⇓ ≠

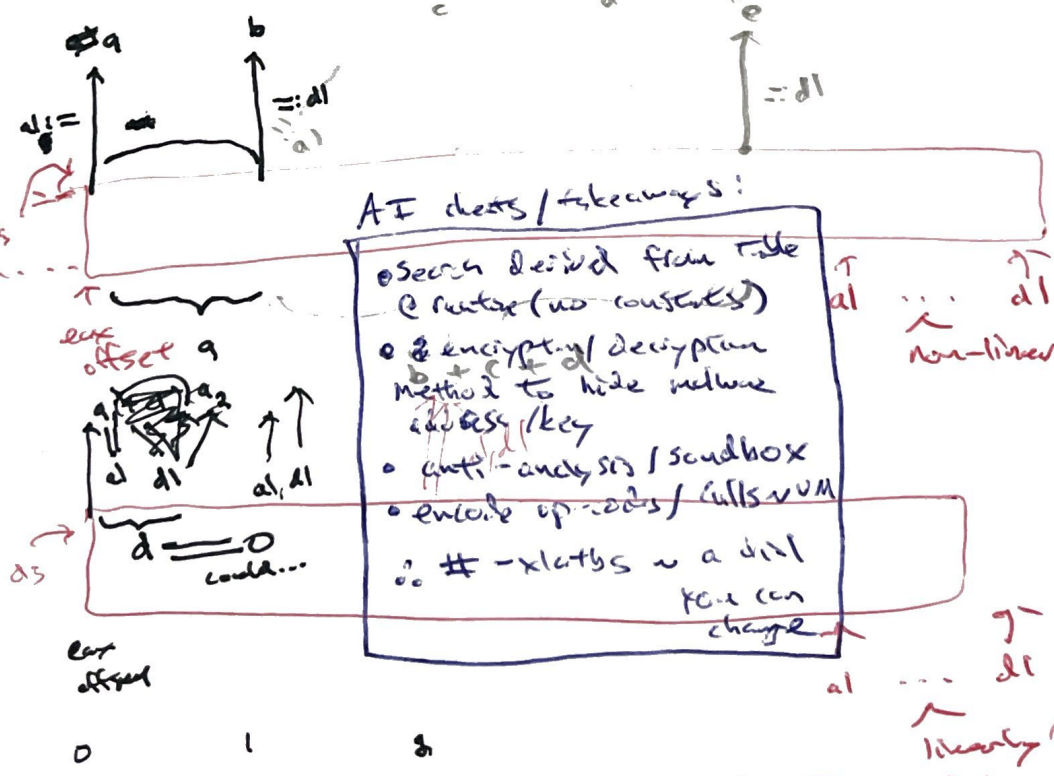
al ~ dl

dl == 0 → EXIT

ELSE loop until al = dl

pattern?

jumps by precising memory value & checking if they are same



AI checks / takeaways:

- search derived from table @ runtime (no constants)
- & encrypting / decryption method to hide malware address / key
- anti-analysis / sandbox
- encode opcodes / calls w/ VM
- ∴ # -xlatbs ~ a dial you can change

∴ different ways to step through a lookup table.

adjacent modify table values